C Programming - Assignment 15

Q.1 Implement Singly Linear Linked List

Problem Statement

Write a program to implement a **Singly Linear Linked List** and perform the following operations:

- AddFirst
- AddLast
- AddPosition
- DeleteFirst
- DeleteLast
- DeletePosition
- Traverse (Display)

Concepts Used

- Linked List
- Dynamic Memory Allocation (`malloc()`, `free()`)
- Structures
- Pointers
- Functions
- Traversing Linked List
- Menu-Driven Programming
- Conditional Statements (`if`)
- Looping (`while`)

Step-by-Step Approach

AddFirst

1. Create a new node.
2. Store the data in the new node.
3. Point the new node's `next` to the current head.
4. Update the head to the new node.

AddLast

1. Create a new node.
2. Store the data in the new node.
3. If the list is empty, make the new node the head.
4. Otherwise, traverse to the last node.
5. Link the last node to the new node.

AddPosition

1. Accept the position and data.
2. If the position is 1, call **AddFirst**.
3. Traverse to the node before the given position.
4. Insert the new node between two nodes.

DeleteFirst

1. Check whether the list is empty.
2. Store the head node in a temporary pointer.
3. Move the head to the next node.
4. Free the deleted node.

DeleteLast

1. Check whether the list is empty.
2. Traverse to the last node while keeping track of the previous node.
3. Remove the last node.
4. Free the deleted node.

DeletePosition

1. Accept the position.
2. If the position is 1, call **DeleteFirst**.
3. Traverse to the specified position.
4. Update the links to remove the node.
5. Free the deleted node.

Traverse (Display)

1. Check whether the list is empty.
2. Start from the head node.
3. Display each node's data.
4. Continue until `NULL` is reached.

Test Cases

Test Case 1 – AddFirst

Input

```
AddFirst(10)
AddFirst(20)
AddFirst(30)
```

Output

```
30 -> 20 -> 10 -> NULL
```

Test Case 2 – AddLast

Input

```
AddLast(10)
AddLast(20)
AddLast(30)
```

Output

```
10 -> 20 -> 30 -> NULL
```

Test Case 3 – AddPosition

Input

```
Initial List:
10 -> 20 -> 40

AddPosition(3,30)
```

Output

```
10 -> 20 -> 30 -> 40 -> NULL
```

Test Case 4 – DeleteFirst

Input

```
10 -> 20 -> 30 -> NULL
```

Output

```
20 -> 30 -> NULL
```

Test Case 5 – DeleteLast

Input

```
10 -> 20 -> 30 -> NULL
```

Output

```
10 -> 20 -> NULL
```

Test Case 6 – DeletePosition

Input

```
10 -> 20 -> 30 -> 40 -> NULL

DeletePosition(3)
```

Output

```
10 -> 20 -> 40 -> NULL
```

Test Case 7 – Traverse (Display)

Input

```
10 -> 20 -> 30 -> 40 -> NULL
```

Output

```
10 -> 20 -> 30 -> 40 -> NULL
```
